

Applicazioni Biomediche delle CNN

6.6.1 Dalla Classificazione alla Segmentazione

Le CNN viste finora **classificano** un'intera immagine:

Input: immagine → Output: classe (singolo valore)

In campo biomedico spesso serve la **segmentazione semantica**:

Input: immagine → Output: mappa di classi (stessa dimensione dell'input)

Ogni pixel viene classificato in una delle K classi (es. background, organo, lesione).

□ **Info:** Relazione con Segmentazione Classica

Questo approccio basato su deep learning si affianca ai metodi classici di segmentazione come K-means, FCM e Region Growing, Level Set, Watershed. I metodi DL eccellono quando si hanno sufficienti dati annotati; i metodi classici sono preferibili per problemi ben definiti o con pochi dati.

6.6.2 Object Detection

Se l'immagine contiene **più oggetti** distinti, prima della classificazione è necessaria la **object detection**.

6.6.2.1 Approccio Semplice: Cropping

Se gli oggetti sono pochi e localizzati:

- Definire regioni rettangolari (bounding box)
- Classificare ogni regione separatamente

6.6.2.2 Approcci Avanzati

- **R-CNN**: regioni proposte + classificazione
 - **YOLO**: detection in un singolo passaggio
 - **Faster R-CNN**: Region Proposal Network integrata
-

6.6.3 Semantic Segmentation con CNN

6.6.3.1 Il Problema

Una CNN classica **riduce** le dimensioni dell'immagine:

$$H \times W \times C \rightarrow 1 \times 1 \times K$$

Per la segmentazione servono dimensioni **uguali** all'input:

$$H \times W \times C \rightarrow H \times W \times K$$

6.6.3.2 L'Idea dell'Autoencoder

Un **autoencoder** ha due parti:

1. **Encoder**: comprime l'input (come una CNN classica)
2. **Decoder**: ricostruisce l'immagine dalla rappresentazione compressa

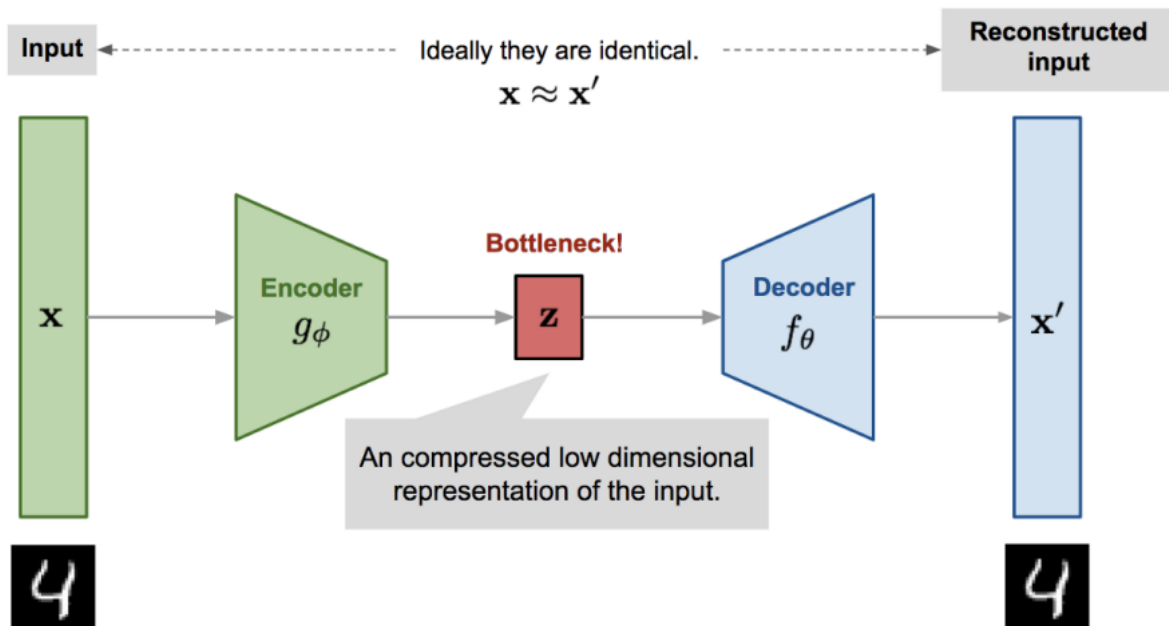


Figura 1: Pasted image 20251201192321.png

□ **Attenzione:** Problema

La compressione causa **perdita di dettaglio** spaziale, inadatta per segmentazione accurata.

6.6.4 U-Net

La **U-Net** (Ronneberger et al., 2015) è l'architettura di riferimento per la segmentazione di immagini biomediche.

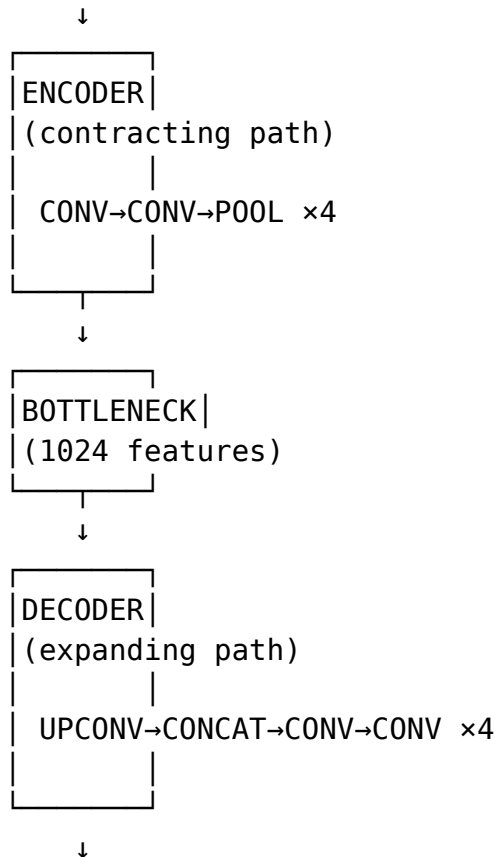
6.6.4.1 Caratteristiche Distintive

1. **Architettura a U** simmetrica
2. **Skip connections** tra encoder e decoder

3. Progettata per funzionare con **pochi dati** di training

6.6.4.2 Architettura

Input (572×572×1)



Output (388×388×K)

6.6.4.3 Encoder (Contracting Path)

Simile a una CNN classica:

- Blocchi: **CONV → ReLU → CONV → ReLU → MaxPool**
- Ogni blocco raddoppia il numero di feature
- MaxPool dimezza le dimensioni spaziali

6.6.4.4 Decoder (Expanding Path)

Ricostruisce le dimensioni:

- **Up-convolution** (transposed convolution): raddoppia le dimensioni
- **Concatenazione** con feature dell'encoder (skip connection)
- **CONV → ReLU → CONV → ReLU**

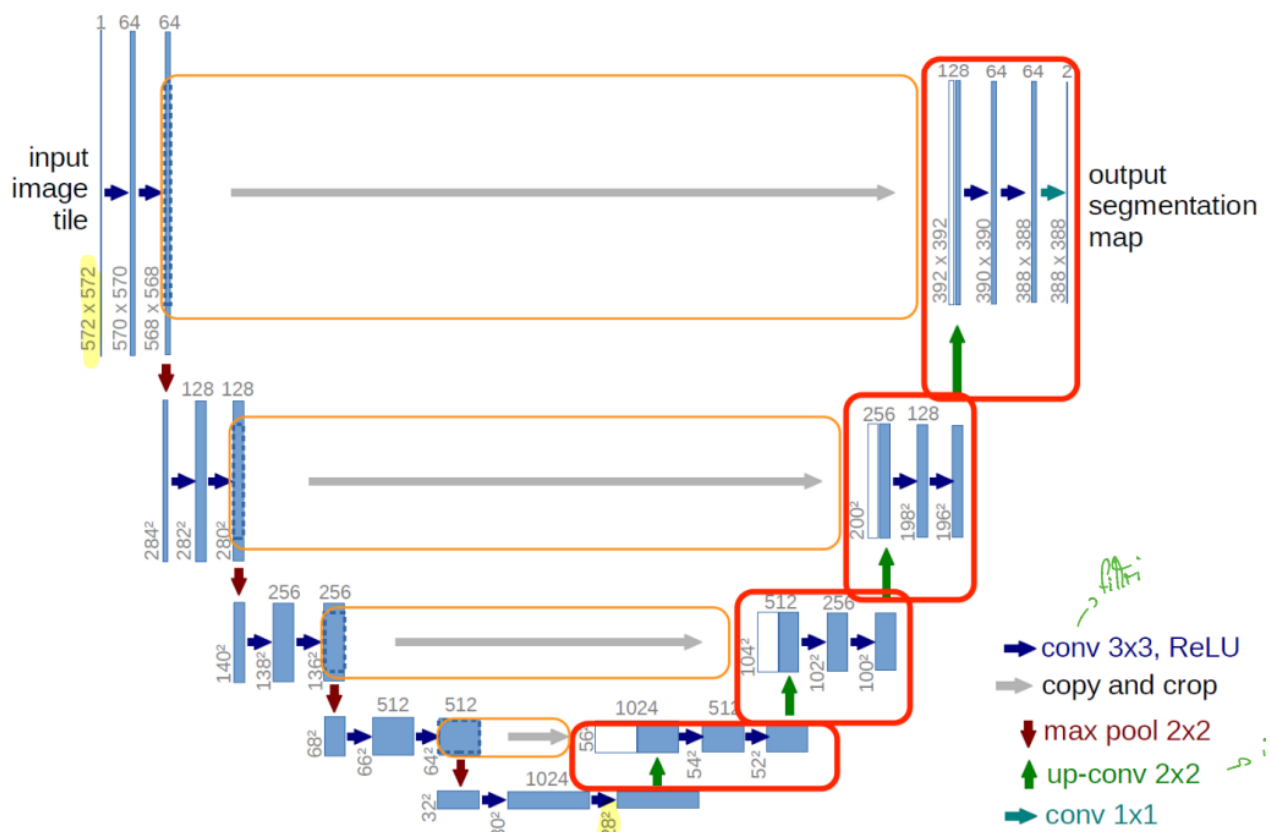


Figura 2: Pasted image 20251201192336.png

6.6.4.5 Skip Connections

Le **skip connections** sono fondamentali:

- Collegano layer dell'encoder ai corrispondenti del decoder
- Preservano i **dettagli spaziali** persi nella compressione
- Permettono al decoder di usare feature a **diverse risoluzioni**
 - **Suggerimento:** Intuizione

L'encoder cattura il "cosa" (feature semantiche), le skip connections aggiungono il "dove" (localizzazione precisa).

6.6.5 Implementazione U-Net

6.6.5.1 MATLAB

```
% Definizione semplificata U-Net
layers = [
    % Encoder
    imageInputLayer([256 256 1])
    convolution2dLayer(3,64,'Padding','same')
    reluLayer
    convolution2dLayer(3,64,'Padding','same')
    reluLayer
    maxPooling2dLayer(2,'Stride',2)

    % ... blocchi encoder aggiuntivi ...

    % Bottleneck
    convolution2dLayer(3,512,'Padding','same')
    reluLayer

    % Decoder (con skip connections)
    transposedConv2dLayer(2,256,'Stride',2)
    % concatenazione con encoder features
    convolution2dLayer(3,256,'Padding','same')
    reluLayer

    % ... blocchi decoder aggiuntivi ...

    % Output
    convolution2dLayer(1,K) % K classi
    softmaxLayer
```

```
pixelClassificationLayer()  
];
```

6.6.5.2 Loss Function per Segmentazione

Cross-entropy per pixel:

$$L = -\frac{1}{N_{pixel}} \sum_{i,j} \sum_k t_{ijk} \log(p_{ijk})$$

Dove:

- t_{ijk} = ground truth (one-hot) per pixel (i, j) classe k
- p_{ijk} = probabilità predetta

Dice Loss (più robusta a sbilanciamento classi):

$$L_{Dice} = 1 - \frac{2|P \cap G|}{|P| + |G|}$$

6.6.6 Metriche di Valutazione per Segmentazione

6.6.6.1 Pixel Accuracy

$$\text{Accuracy} = \frac{\text{pixel corretti}}{\text{pixel totali}}$$

□ **Attenzione:** Problema

Con classi sbilanciate (es. 95% background), accuracy alta non indica buona segmentazione.

6.6.6.2 IoU (Intersection over Union)

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|}$$

Dove P = predizione, G = ground truth.

6.6.6.3 Dice Coefficient

$$\text{Dice} = \frac{2|P \cap G|}{|P| + |G|} = \frac{2 \times \text{IoU}}{1 + \text{IoU}}$$

□ **Info:** Relazione con Validazione

Queste metriche corrispondono a quelle viste nel capitolo sulla Validazione degli Algoritmi.

6.6.7 Applicazioni Biomediche

6.6.7.1 Segmentazione di Organi

- **Cuore:** ventricoli, atri, miocardio
- **Cervello:** sostanza grigia/bianca, lesioni
- **Fegato:** parenchima, vasi, tumori
- **Polmoni:** parenchima, noduli

6.6.7.2 Segmentazione di Lesioni

- Tumori
- Placche aterosclerotiche
- Lesioni traumatiche

6.6.7.3 Quantificazione

Dalla segmentazione si possono calcolare:

- **Volumi** degli organi/lesioni
 - **Forma** (sfericità, compattezza)
 - **Texture** (analisi regionale)
-

6.6.8 Varianti di U-Net

6.6.8.1 U-Net++

- Skip connections **dense** a più livelli
- Migliore fusione di feature multi-scala

6.6.8.2 Attention U-Net

- **Attention gates** nelle skip connections
- Focus automatico sulle regioni rilevanti

6.6.8.3 3D U-Net

- Per volumi (CT, MRI 3D)
- Stesso principio, operazioni 3D

6.6.8.4 nnU-Net

- Framework **auto-configurante**
- Sceglie automaticamente preprocessing, architettura, training

- State-of-the-art su molti benchmark biomedici
-

6.6.9 Sfide nell'Imaging Biomedico

6.6.9.1 Dataset Limitati

- **Attenzione:** Il Problema Fondamentale

I dataset biomedici sono tipicamente molto più piccoli di ImageNet.

Soluzioni:

- Data augmentation aggressiva
- Transfer learning
- Semi-supervised learning
- Active learning

6.6.9.2 Annotazioni Costose

L'annotazione pixel-wise richiede **esperti clinici** → molto costosa.

Soluzioni:

- Weak supervision (label a livello di immagine)
- Self-supervised pretraining

6.6.9.3 Variabilità

Immagini da scanner/protocolli diversi hanno caratteristiche diverse.

Soluzioni:

- Domain adaptation
- Harmonization
- Training multi-centro

6.6.9.4 Interpretabilità

In clinica, serve capire **perché** la rete ha fatto una certa predizione.

Soluzioni:

- Grad-CAM (activation visualization)
 - Uncertainty quantification
 - Explainable AI
-

6.6.10 Risorse e Tool

6.6.10.1 Framework

- **MONAI** (Medical Open Network for AI): framework PyTorch per medical imaging
- **nnU-Net**: framework auto-configurante
- **TorchIO**: preprocessing e augmentation per volumi 3D

6.6.10.2 Dataset Pubblici

Dataset	Modalità	Task
BRATS	MRI	Tumori cerebrali
LIDC-IDRI	CT	Noduli polmonari
ACDC	MRI cardiaca	Segmentazione cuore
ISIC	Dermatoscopia	Lesioni cutanee